

Entwurf digitaler Systeme

5. Kombinatorische Schaltungen

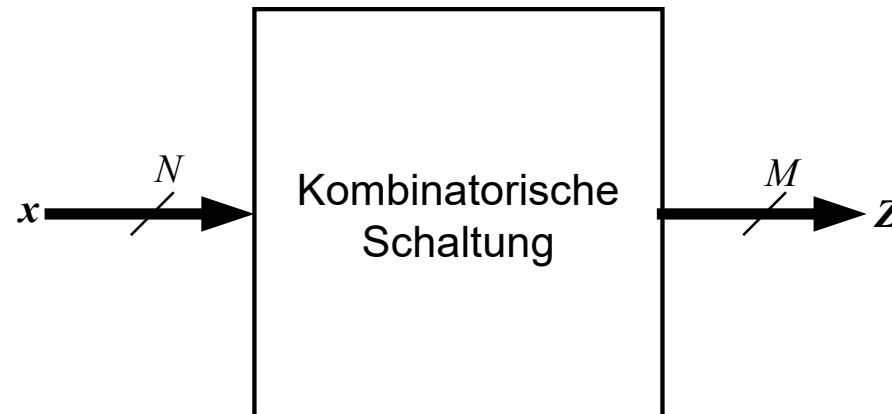
5.1 Einführung

- Kombinatorische Schaltungen werden in die „klassischen“ zweistufigen Schaltungen, aber auch in mehrstufige Schaltungen unterteilt.
- Beide Schaltungskonzepte haben ihre Vor- und Nachteile, auf die jedoch erst im Laufe der Abschnitte näher eingegangen werden soll.

- **Definition 5.1** (Kombinatorische Schaltung)
 - Eine kombinatorische Schaltung ist ein digitales System, bestehend aus logischen Verknüpfungen, das ein Eingangsbündel oder einen Eingangsvektor (Matrix) mit Hilfe einer Kombinationsmatrix K in ein Ausgangsbündel oder einen Ausgangsvektor (Matrix) abbildet. Kombinatorische Schaltungen besitzen keine internen Rückführungen und kein Speicherverhalten.

5.1 Einführung

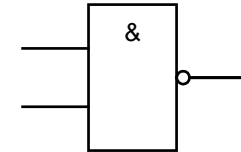
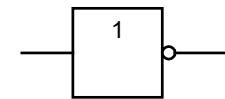
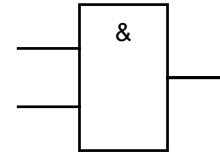
■ Einordnung



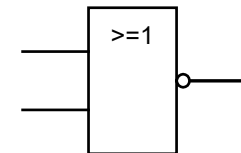
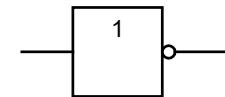
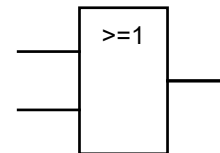
5.1 Einführung

- Basissysteme
 - Wie Sie bereits erkannt haben, lässt sich jede beliebige kombinatorische Schaltung aus vier so genannten Basissystemen logischer Grundfunktionen realisieren.

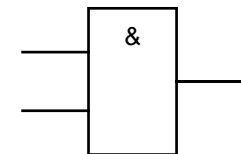
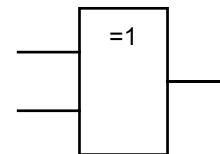
AND-Basissystem



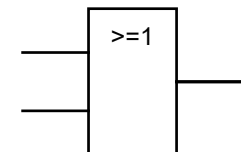
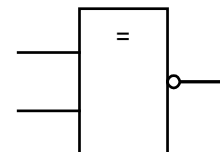
OR-Basissystem



XOR-Basissystem



XNOR-Basissystem



5.2 Realisierung zweistufiger kombinatorischer Schaltungen

- Für zweistufige kombinatorische Schaltungen verwendet man eine Realisierungsstrategie mit Hilfe der KNF oder der DNF.
- Etwaige Negierungen der Eingangsvariablen werden immer mittels eines vorgeschalteten Inverters durchgeführt, wobei die Schaltung genau genommen nicht mehr zweistufig ist, sondern dreistufig.
- Da jedoch die Negation sich nicht auf alle Variablen bezieht, oder von anderen Gattern bereitgestellt werden kann, werden die Inverter nicht mit in die Stufenzahl mit einbezogen.

5.2.1 Codierungsschaltungen

- In diesem Abschnitt werden verschiedene Codierungsschaltungen vorgestellt, die in der Praxis – entweder diskret aufgebaut oder in ICs – oft Verwendung finden.
- Streng genommen müsste die Theorie dieser Codes in einem gesonderten Kapitel zur Zahlentheorie behandelt werden.
- Da jedoch die Schaltungsrealisierung im Vordergrund steht, soll darauf, zu Gunsten des Praktikums und einiger praktischer Beispiele, verzichtet werden.
- **Definition 5.2** (Code)
 - Ein Code ist eine Zuordnungsvorschrift, die für ein Zeichen aus einem Zeichenvorrat eine Bitkombination festlegt.

5.2.1 Codierungsschaltungen

- Man unterscheidet zwischen *alphanumerischen* und *numerischen Codes*.
 - Mit **ersteren** lassen sich Ziffern, Buchstaben und Sonderzeichen binär verschlüsseln. Zur ausschließlichen Codierung von Ziffern verwendet man **numerische** Codes.
 - Numerische Codes unterscheiden noch zwischen *Wort- und Zifferncodes*. Der Wortcode verschlüsselt Zahlen, während der Zifferncode einzelne Ziffern codiert.
- Das wohl bekannteste Beispiel ist der *ASCII-Code* (American Standard Code for Information Interchange).
 - Dieser 7-bit-Code wird üblicherweise in der Rechnertechnik zur Eingabe von Zahlen und Buchstaben sowie Sonder- und Steuerzeichen verwendet.
 - Da in der Regel in dieser Anwendung immer ein Byte übertragen wird, ist auch eine erweiterte Version mit dem doppelten Zeichensatzvorrat (8 Bits) verfügbar. Es werden zuzüglich grafische Symbole übertragen.

5.2.1.1 Wortcodes

- Der bekannteste Wortcode ist der *Dualcode*. Jede Dezimalzahl wird in eine entsprechende Dualzahl durch Gewichtung der einzelnen Stellen umgewandelt. Die Regeln hierzu sind Ihnen aus der ersten Lerneinheit bekannt.
- Ein in der Praxis häufig verwendeter, ungewichteter Code ist der *Gray-Code*. In der ersten Lerneinheit wurde er zur Erzeugung des K-Plans verwendet.
 - Bekanntermaßen wurde er so aufgebaut, dass sich die einzelnen nacheinander folgenden Wörter nur um jeweils eine Binärstelle ändern. Dieses hat in der Praxis den Vorteil, dass eine Fehlererkennung möglich wird, da die Schrittweite (1 Bitstelle) bekannt ist.
- Beispielsweise werden in *Drehgebern* (Inkrementalgeber / Absolutwertgeber) derartige Codierungen eingesetzt, um sicherzustellen, dass eine korrekte Winkelübertragung gewährleistet ist.

5.2.1.2 Zifferncodes

- **Zifferncodes** werden zur Ein- und Ausgabe in digitalen Systemen verwendet, da in der Regel mit dezimalen Zahlen (Ziffern) operiert wird.
- Von besonderem Interesse sind deshalb verschiedene Varianten des **BCD-Codes** (*Binary Coded Decimal*). Für die Eingabe von dezimalen Zahlen (0 bis 9) sind pro Stelle vier Bits notwendig.
 - Diese Bits werden *Tetrad*e genannt [(gr.): tetratos – eine Ordinalzahl, z. B.: Die Vierte].
 - Andere gebräuchliche Begriffe sind *Nibble* oder *Quadrupel*.

5.2.1.2 Zifferncodes

■ 8-4-2-1-Code

- Die ersten 10 Zahlenkombinationen der Vier-Bit-Binärzahlen werden für die Darstellung der dezimalen Ziffern 0 bis 9 verwendet. Alle anderen Binärkombinationen (10-15, $A_{\text{hex}} - F_{\text{hex}}$) sind nicht zulässig und dürfen keinesfalls verwendet werden. Diese werden *Pseudotetraden* (*PT*) genannt.
 - Die Dezimalzahl 2005 wird im 8-4-2-1-Code durch (0010 0000 0000 0101) dargestellt.

5.2.1.2 Zifferncodes

■ BCD-Gray-Code

- Binär codierte Dezimalzahlen werden mittels der ersten 10 Tetraden des Gray-Codes dargestellt. Die restlichen Tetraden werden wiederum als Pseudotetraden aufgefasst. Der Verwendung des BCD-Gray-Codes liegt wiederum die Tatsache zugrunde, eine möglichst eindeutige und störsichere Bearbeitung zu gewährleisten.

■ 3-Excess-Code oder Stibitz-Code

- Man erhält die Darstellung einer Zahl, indem man zur Binärdarstellung 0011 (= 3) hinzuaddiert. Der Excess-3-Code kann mit einer einfachen Korrektur auch zum Rechnen verwendet werden. Hierzu muss bei der Addition ein Übertrag erfolgen, falls ein Zehnerübertrag stattfindet (0011 = 3 hinzuaddieren). Findet kein Übertrag statt, so muss das Ergebnis um 1101 (= -3) korrigiert werden. Die Werte 1111 und 0000 treten nicht auf, was ein Vorteil für die Nachrichtentechnik ist, da diese Worte leicht bei einem Fehlverhalten auftreten. Der Stibitz-Code ist selbstkomplementierend und ermöglicht eine Unterscheidung von gerade/ungerade sowie die Rundungserkennung ($<$ oder > 5). Er wird hauptsächlich in der Rechnertechnik im Automatisierungsbereich verwendet. Die nicht verwendeten Kombinationen stellen Pseudotetraden dar.

5.2.1.2 Zifferncodes

■ Aiken-Code

- Der Aiken-Code ist derart gestaltet, dass durch eine einfache Invertierung aller Binärstellen eine Komplementbildung möglich wird. Die invertierte Darstellung entspricht dem Neuner-Komplement der ursprünglichen Dezimalzahl. Die nicht verwendeten Kombinationen entsprechen den Pseudotetraden. Dieser Code wurde in der Rechnertechnik verwendet. Er kommt heute nur noch zum Einsatz bei: Digitaluhren, Taschenrechnern, ...
- Gewichtung: 2-4-2-1

5.2.1.3 Spezielle Ein- und Ausgabeumsetzer

- Vielfach müssen binäre Zahlen mit Hilfe von Tasten oder Schaltern eingegeben werden.
- Dazu verwendet man so genannte
 - *1-aus-N-Coder.*
- Ebenso lassen sich
 - *1-aus-N-Decoder*
realisieren, mit deren Hilfe Baugruppen wie Relais, Lampen usw. anhand einer Codierung ansteuerbar sind.

5.2.1.3 Spezielle Ein- und Ausgabeumsetzer

■ 1-aus-N-Coder

- Ein 1-aus-N-Coder ist eine kombinatorische Schaltung mit N Eingängen und $\text{Id}(N)$ Ausgängen.
- Im Fall einer dezimalen Codierung besitzt der Coder 16 Eingänge, von denen 10 genutzt werden und $\text{Id}(16) = 4$ binär codierte Ausgänge. Pseudotetraden werden nicht berücksichtigt.
- Diese Schaltung wird als 1-aus-10-Coder bezeichnet.

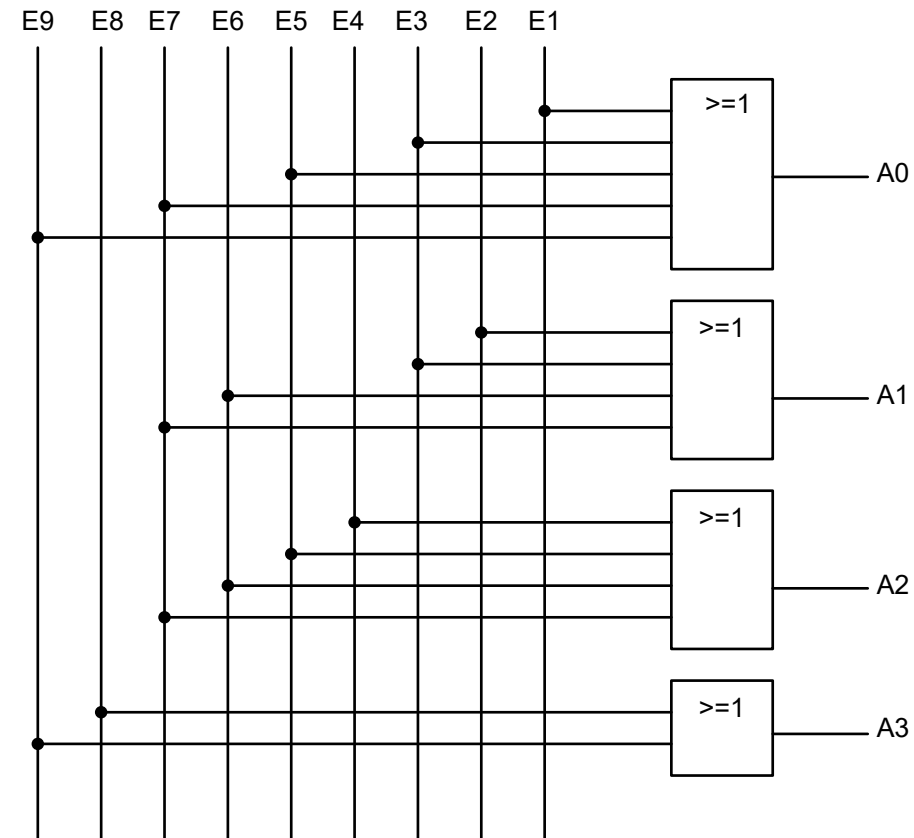
■ 1-aus-N-Decoder

- Ein 1-aus-N-Decoder ist eine kombinatorische Schaltung mit N Ausgängen und $\text{Id}(N)$ Eingängen.
- Im Fall einer dezimalen Codierung besitzt der Decoder 16 Ausgänge, von denen 10 genutzt werden und $\text{Id}(16) = 4$ binär codierte Eingänge. Pseudotetraden werden nicht berücksichtigt.
- Die Schaltung wird als 1-aus-10-Decoder bezeichnet.

5.2.1.3 Spezielle Ein- und Ausgabeumsetzer

■ Beispiel: 1-aus-10 – 8-4-2-1-Coder

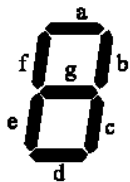
i	Eingänge										Ausgänge			
	E9	E8	E7	E6	E5	E4	E3	E2	E1	E0	A ₃	A ₂	A ₁	A ₀
0	0	0	0	0	0	0	0	0	0	1	0	0	0	0
1	0	0	0	0	0	0	0	0	1	0	0	0	0	1
2	0	0	0	0	0	0	0	1	0	0	0	0	1	0
3	0	0	0	0	0	0	1	0	0	0	0	0	1	1
4	0	0	0	0	0	1	0	0	0	0	0	1	0	0
5	0	0	0	0	1	0	0	0	0	0	0	1	0	1
6	0	0	0	1	0	0	0	0	0	0	0	1	1	0
7	0	0	1	0	0	0	0	0	0	0	0	1	1	1
8	0	1	0	0	0	0	0	0	0	0	1	0	0	0
9	1	0	0	0	0	0	0	0	0	0	1	0	0	1



5.2.1.3 Spezielle Ein- und Ausgabeumsetzer

■ 7-Segment-Decoder

- Eine vielfach verwendete Schaltung stellt der 7-Segment-Decoder dar. Er dient zur Anzeige dezimaler Zahlenwerte mittels einer 7-Segment-Anzeige. Die Schaltung findet vielfältige Anwendung in allen Arten digitaler Systeme.



dez	D	C	B	A	a	b	c	d	e	f	g
0	0	0	0	0	1	1	1	1	1	1	0
1	0	0	0	1	0	1	1	0	0	0	0
2	0	0	1	0	1	1	0	1	1	0	1
3	0	0	1	1	1	1	1	1	0	0	1
4	0	1	0	0	0	1	1	0	0	1	1
5	0	1	0	1	1	0	1	1	0	1	1
6	0	1	1	0	1	0	1	1	1	1	1
7	0	1	1	1	1	1	1	0	0	0	0
8	1	0	0	0	1	1	1	1	1	1	1
9	1	0	0	1	1	1	1	1	0	1	1

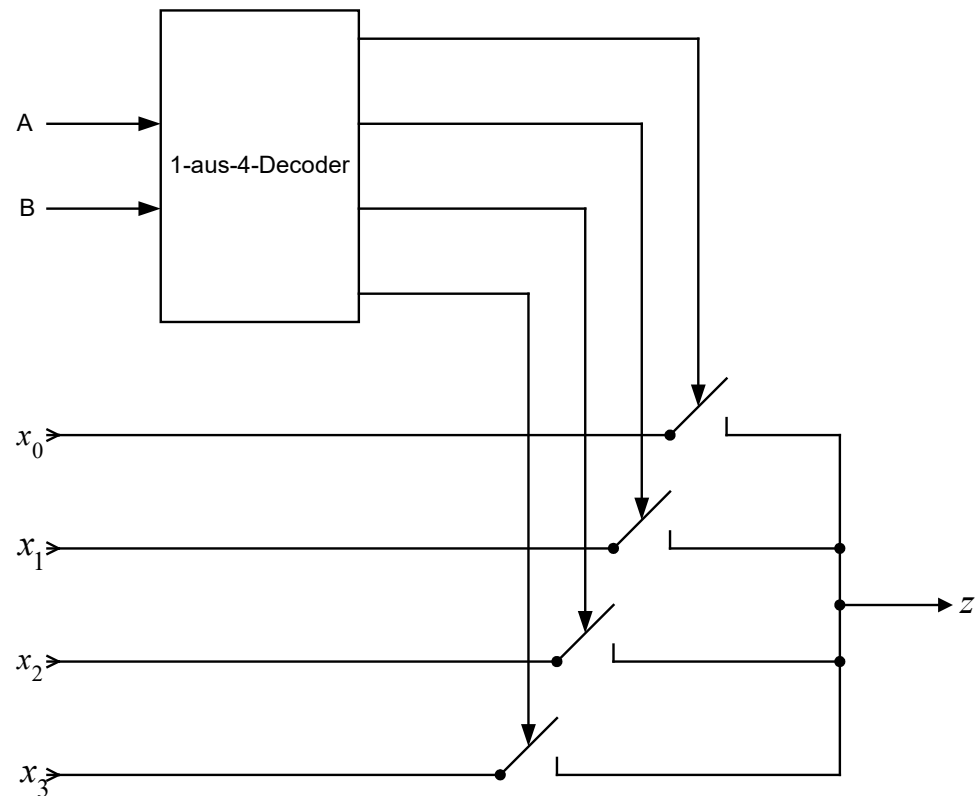
5.2.2 Kombinatorische Grundsaltungen

- Im nächsten Abschnitt befassen wir uns mit einigen typischen Grundsaltungen, die in der Digitaltechnik immer wieder Verwendung finden.
- Diese können sowohl mit einfachen Logikbausteinen diskret aufgebaut sein, oder als Grundsaltungen in applikationsspezifischen Schaltungen verwendet werden.
- Aus ihnen lassen sich weitere, dann jedoch komplexere, Schaltungen generieren, die **in der Praxis von großer Bedeutung** sind.

5.2.2 Kombinatorische Grundsaltungen

■ Multiplexer

- Ein Multiplexer schaltet eine Anzahl von N Signalen (Datenleitungen) auf einen Ausgang. Hierzu wird zum Schalten ein 1-aus- N -Decoder verwendet. Er wählt von N Eingängen denjenigen aus, dessen Adresse mit der eingegebenen übereinstimmt. Der entsprechende Eingang wird auf den Ausgang geschaltet.



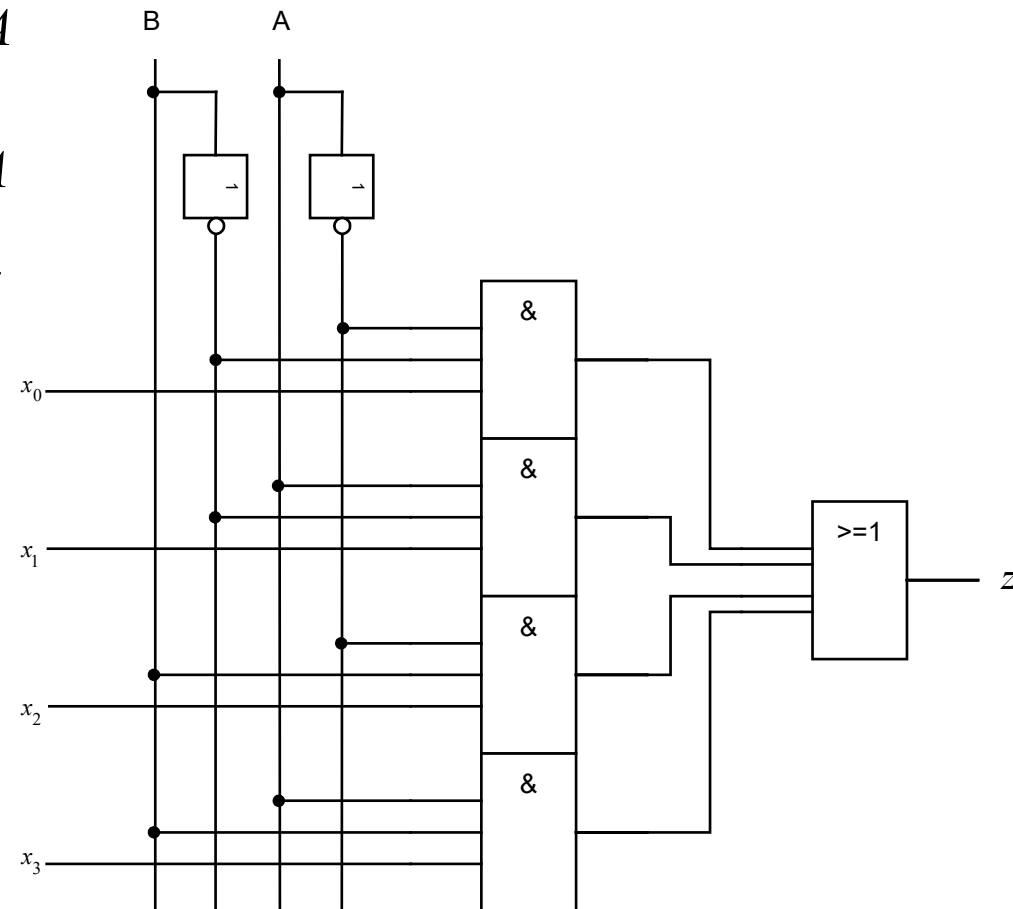
5.2.2 Kombinatorische Grundschaltungen

■ Digitale Schaltung eines 4-fach-Multiplexers

- Schalter 0 geschlossen (00): $\overline{B} \cdot \overline{A}$
- Schalter 1 geschlossen (01): $\overline{B} \cdot A$
- Schalter 2 geschlossen (10): $B \cdot \overline{A}$
- Schalter 3 geschlossen (11): $B \cdot A$



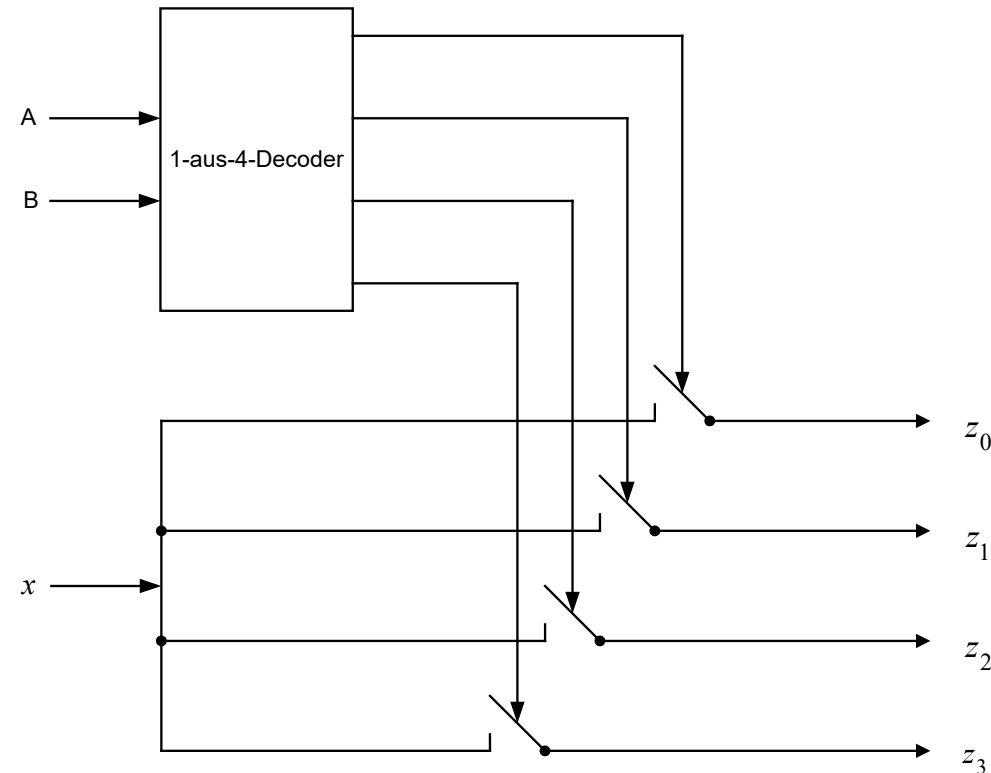
$$Z = \overline{B} \cdot \overline{A} \cdot x_0 + \overline{B} \cdot A \cdot x_1 + B \cdot \overline{A} \cdot x_2 + B \cdot A \cdot x_3$$



5.2.2 Kombinatorische Grundsaltungen

■ Demultiplexer

- Mit Hilfe eines Demultiplexers wird ein Eingang an mehrere Ausgänge verteilt. Hierzu wird zum Schalten ein 1-aus-N-Decoder verwendet. Er wählt von N Ausgängen denjenigen aus, dessen Adresse mit der eingegebenen übereinstimmt. Der entsprechende Eingang wird auf den Ausgang geschaltet.



5.2.2 Kombinatorische Grundschaltungen

■ Digitale Schaltung eines 4-fach-Demultiplexers

- Schalter 0 geschlossen (00): $\overline{B} \cdot \overline{A}$
- Schalter 1 geschlossen (01): $\overline{B} \cdot A$
- Schalter 2 geschlossen (10): $B \cdot \overline{A}$
- Schalter 3 geschlossen (11): $B \cdot A$

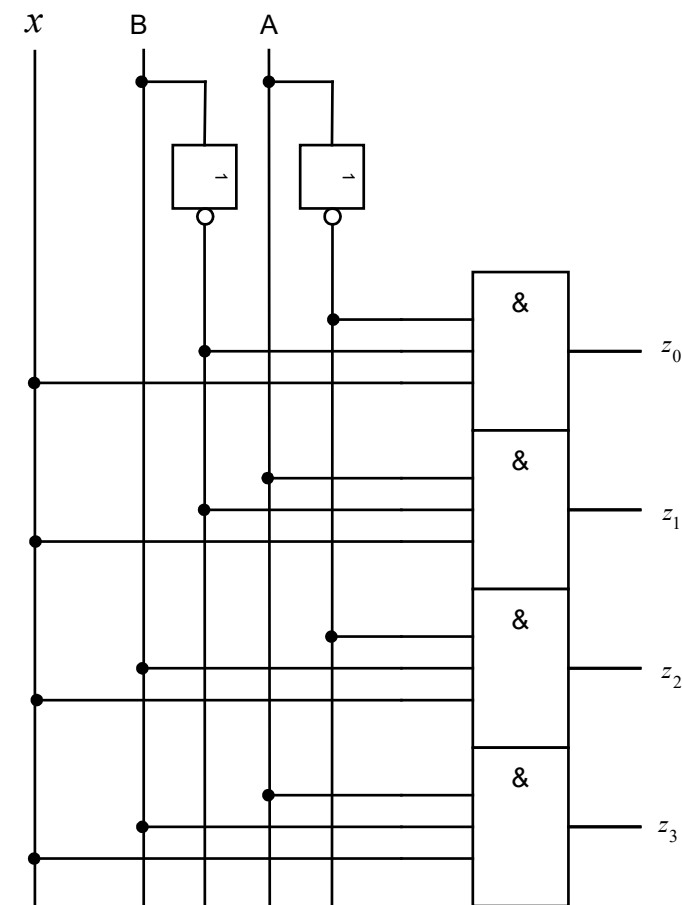


$$z_0 = \overline{B} \cdot \overline{A} \cdot x$$

$$z_1 = \overline{B} \cdot A \cdot x$$

$$z_2 = B \cdot \overline{A} \cdot x$$

$$z_3 = B \cdot A \cdot x$$



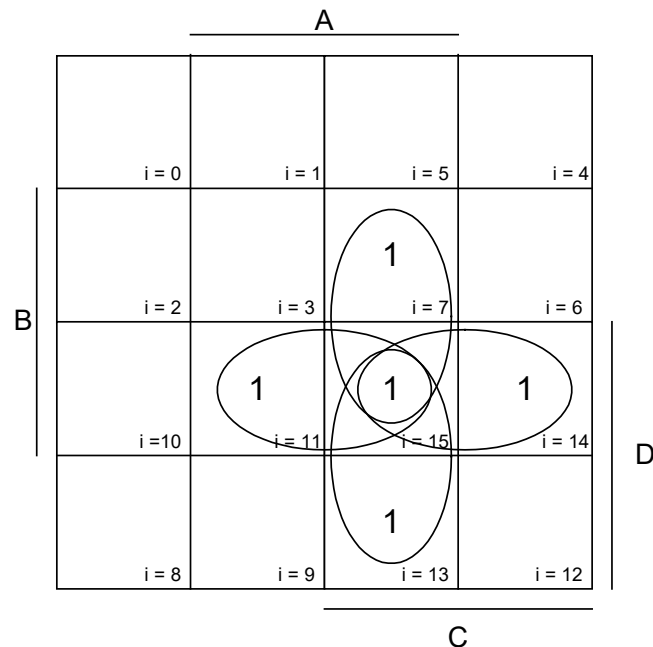
5.2.2 Kombinatorische Grundsaltungen

■ Majoritätsschaltung (Majority Logic)

- Eine Majoritätsschaltung liefert an ihrem Ausgang nur dann eine logische 1, wenn die Mehrheit [lat: majoritas] der Eingangsvariablen den Wert 1 zeigen.
- Die Mehrheit ist dann erreicht, wenn mehr als die Hälfte der Variablen den Wert 1 besitzen.
- Das Symbol für eine derartige Schaltung ist: $\left(> \frac{N}{2} \right)$.

5.2.2 Kombinatorische Grundsaltungen

- Beispiel Majoritätslogik für $N = 4$



$$Z = CBA + DCA + DBA + DCB$$

	Eingang				
Dezimal	D	C	B	A	Z
0	0	0	0	0	0
1	0	0	0	1	0
2	0	0	1	0	0
3	0	0	1	1	0
4	0	1	0	0	0
5	0	1	0	1	0
6	0	1	1	0	0
7	0	1	1	1	1
8	1	0	0	0	0
9	1	0	0	1	0
10	1	0	1	0	0
11	1	0	1	1	1
12	1	1	0	0	0
13	1	1	0	1	1
14	1	1	1	0	1
15	1	1	1	1	1

5.2.2 Kombinatorische Grundschaltungen

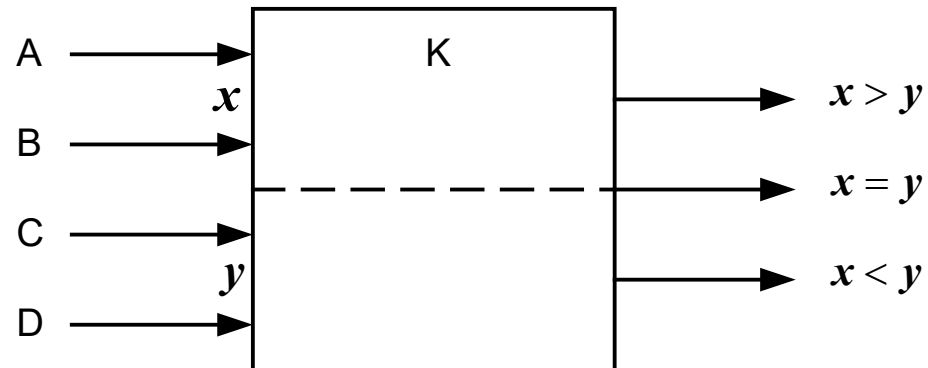
■ Komparator

- Eine eng mit der Majoritätsschaltung verbundene Grundschaltung ist die des Komparators [vergleichen - lat: comparare]. Er dient zum Vergleich zweier Zahlen. Diese Grundschaltung wird oft zur Bestimmung von Auslöse- und Vergleichsschwellen in Steuerungen aller Art verwendet.
- Ein Komparator vergleicht zwei binäre Eingangsvektoren x und y auf Gleichheit. In der Regel besitzt die Schaltung allerdings drei Ausgänge:

$$z_2 : x > y$$

$$z_1 : x = y$$

$$z_0 : x < y$$



Praktikum!

5.2.2 Kombinatorische Grundsaltungen

■ Volladdierer (VA)

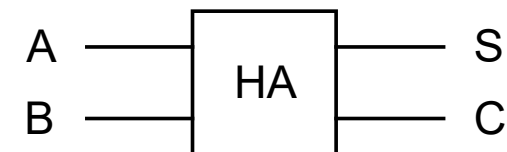
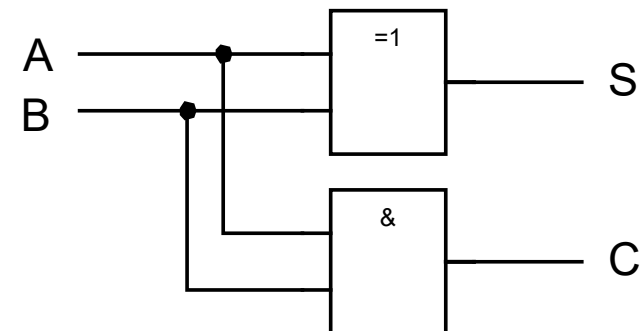
- Eine Addierschaltung addiert *vorzeichenrichtig* zwei binäre Zahlen mit Hilfe einer zweistufigen kombinatorischen Schaltung. Diese besteht im einfachsten Fall aus zwei so genannten *HalbAddierern (HA)*, die entsprechend verschaltet sind.

- Wahrheitstabelle HA, Schaltung und Symbol:

A	B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

$$S = B\bar{A} + \bar{B}A = B \oplus A$$

$$C = BA$$



5.2.2 Kombinatorische Grundschaltungen

■ Volladdierer

- ... nun mit Berücksichtigung des Carry-Out (das kennen wir schon!)

$$S = CBA + C\bar{B}\bar{A} + \bar{C}B\bar{A} + \bar{C}\bar{B}A = C \oplus (B \oplus A).$$

$$O = CB + CA + BA = BA + C(B \oplus A).$$

Umformung:

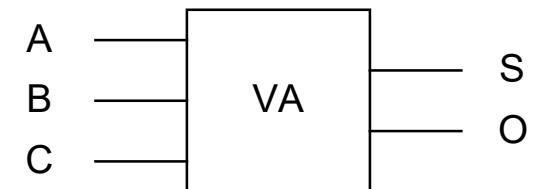
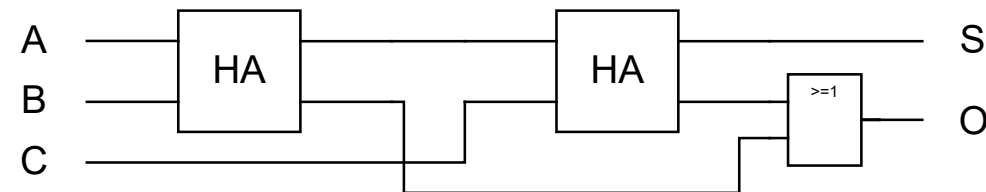
$$S = C(BA + \bar{B}\bar{A}) + \bar{C}(B\bar{A} + \bar{B}A) = \dots$$

$$\dots = C(\overline{B \oplus A}) + \bar{C}(B \oplus A) = \dots$$

$$\dots = C\bar{F} + \bar{C}F = C \oplus F = C \oplus (B \oplus A).$$

$$O = CB + CA + BA = \dots$$

$$\dots = BA + C(B \oplus A).$$



$$O = (A \cdot \bar{B} \cdot C + \bar{A} \cdot B \cdot C) + (A \cdot B \cdot \bar{C} + A \cdot B \cdot C) = \dots$$

$$\dots = (A \oplus B) \cdot C + B \cdot A \cdot (\bar{C} + C)$$



5.2.2 Kombinatorische Grundschaltungen

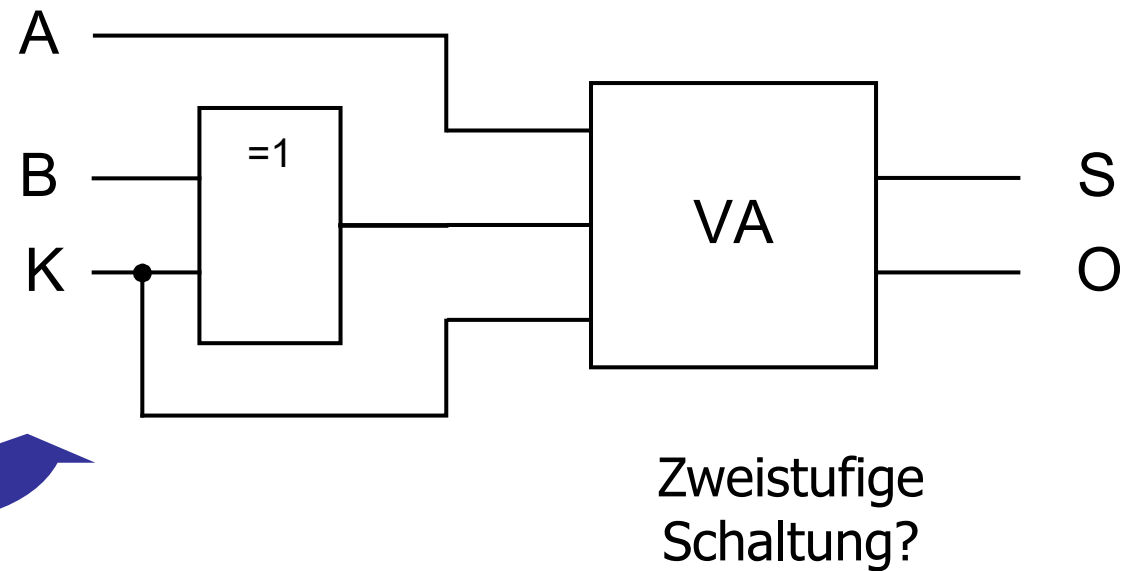
■ Volladdierer

- ... nun als K2-Subtrahierer $x - y = x + (-y)$
- Ansatz Steuertabelle: Wenn $K = 0$ (Kontrollbit) dann $Z = B$; andernfalls $Z = /B$.

B	K	Z
0	0	0
0	1	1
1	0	1
1	1	0

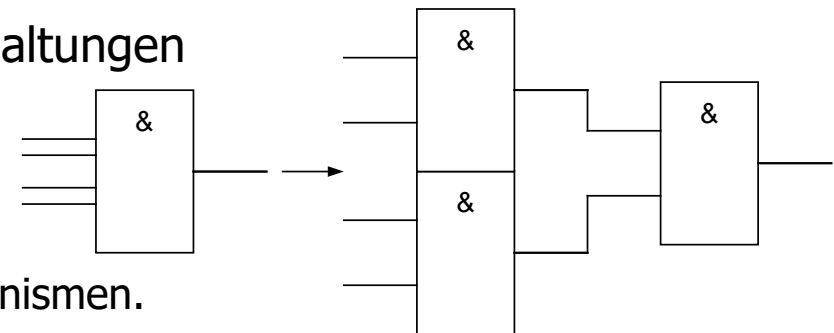
Z: Ausgang

B: Eingang



5.3 Grundlagen mehrstufiger kombinatorischer Schaltungen

- Einerseits haben wir den Formalismus geklärt, wie (zweistufige) kombinatorische Schaltungen realisiert werden, aber bei der Realisierung eines einfachen Volladdierers mit Hilfe von Halbaddierern haben wir gesehen, dass, wenn Sie so wollen, ohne unser Zutun mehrstufige Varianten auf natürliche Weise entstehen.
- Demzufolge muss die Frage gestellt werden, warum dann diese Schaltungen – zunächst – nicht direkt genutzt werden?
- Notwendigkeit zur Nutzung mehrstufiger Schaltungen
 - Gatterbreite (Eingänge) ist begrenzt.
 - Es wird weniger Chipfläche benötigt.
 - Nachteil: Laufzeiten.
 - Nachteil: Aufwand für Datenverteilungsmechanismen.
- Frage nach einem Kostenoptimum (zweistufig / mehrstufig)



5.3 Grundlagen mehrstufiger kombinatorischer Schaltungen

■ Mehrstufige Synthese

- Die Verringerung der Anzahl der Eingänge wird durch Faktorisierung und gleichzeitige Stufenerhöhung erreicht. Es werden die Eigenschaften der Booleschen Algebra genutzt. *Dieser Schritt ist technologieunabhängig.*
- Im zweiten Schritt werden die faktorisierten Gleichungen, unter Verwendung vorhandener Gatter oder Bausteinbibliotheken, auf eine entsprechende Implementierung abgebildet. *Dieser Schritt ist technologieabhängig.*

5.3 Grundlagen mehrstufiger kombinatorischer Schaltungen

- Beispiel (Abb. A, zweistufig)

$$Z = FDA + FEA + FDB + FEB + FDC + FEC + G$$

- Schritt 1 (technologieunabhängig):

$$Z = F(DA + EA + DB + EB + DC + EC) + G = \dots$$

$$\dots = F[A(D + E) + B(D + E) + C(D + E)] + G = \dots$$

$$\dots = F[(A + B + C)(D + E)] + G.$$

- Schritt 2:

- Nun soll angenommen werden, dass ausschließlich Zwei- und Dreifach-Gatter verwendet werden können (technologieabhängig).

Es entsteht folgende Schaltung:

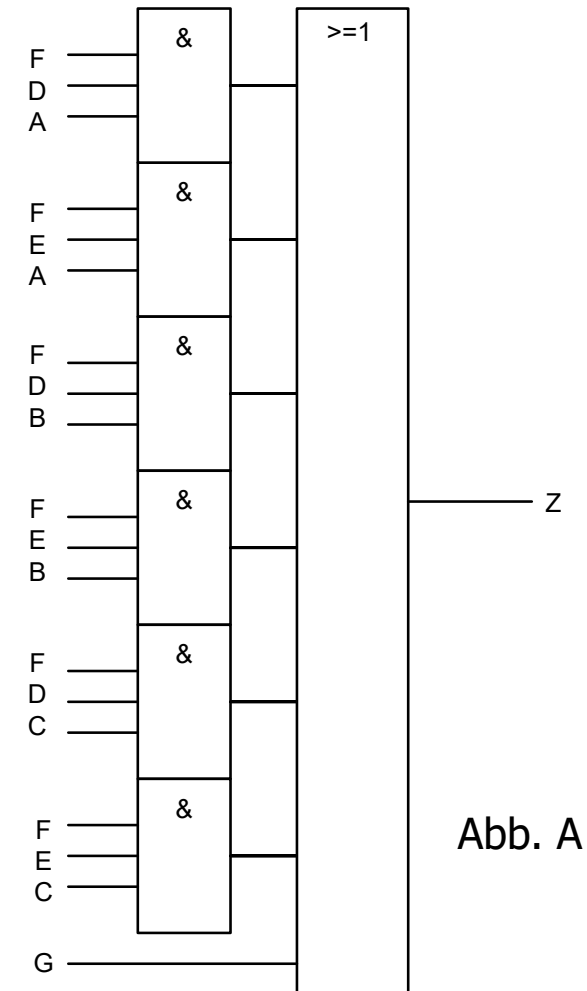
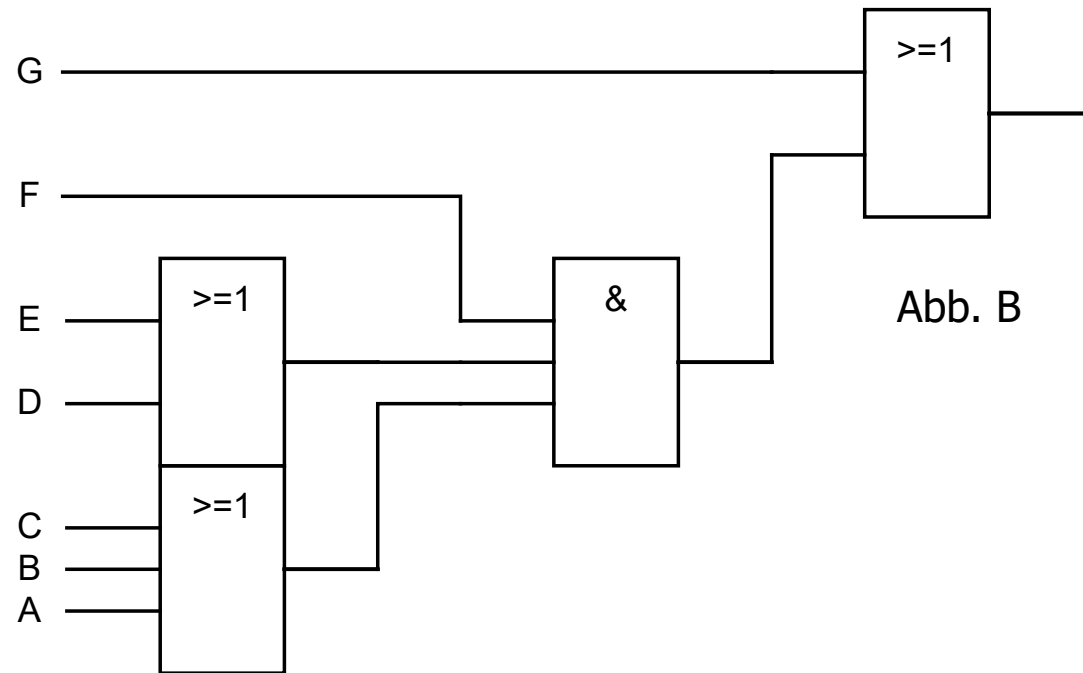


Abb. A

5.3 Grundlagen mehrstufiger kombinatorischer Schaltungen

- Schaltung nach Schritt 2:



- Kosten:

	Abbildung A	Abbildung B
Gatteranzahl	7	4
Literalanzahl	25	10
Stufentiefe	2	3



5.3 Grundlagen mehrstufiger kombinatorischer Schaltungen

- In der praktischen Anwendung gibt es verschiedene Methoden, um optimale mehrstufige Schaltungen zu entwerfen. Zu nennen sind hier u. a. die
 - zergliedernde Faktorisierung und
 - aufbauende Faktorisierung.
- In der Regel muss bei den genannten Verfahren eine KMF oder DMF vorliegen. Auf die Verfahren kann aber im Einzelnen hier nicht weiter eingegangen werden; wir verweisen in diesem Zusammenhang auf die entsprechende Literatur.
- Bei einer mehrstufigen komplexen Schaltung benötigen die Gatter im Vergleich zu der zweistufigen Variante weniger Chipfläche; die Anzahl der Verbindungen und damit die entsprechende Größe der Chipfläche, bzw. der Verdrahtungsaufwand, steigen jedoch. Da die Komplexität der Verbindungen direkt mit der Anzahl der Literale zusammenhängt, versucht man beim mehrstufigen Entwurf diese Größe zu minimieren. Es kann gesagt werden:

Literale = Kosten.
- Es existiert jedoch momentan kein Algorithmus, der das Erreichen einer optimalen Schaltung im Sinne der o. g. Aussage garantiert.

5.4 Dynamisches Verhalten kombinatorischer Schaltungen

- Bis zum jetzigen Zeitpunkt sind kombinatorische Schaltungen im Sinne der Booleschen Algebra betrachtet worden. Das reale Verhalten von Gattern bzw. Schaltungen ist nur angedeutet worden.
- Im nächsten Schritt wollen wir uns von der rein algebraischen Betrachtungsweise lösen und zumindest modellhaft verschiedene Gatter als reale Bauteile auffassen.
- Da es eine Vielzahl verschiedener Bausteintypen mit unterschiedlichem Verhalten gibt, sollen hier nicht verschiedene Technologien (TTL, CMOS, BiCMOS, ECL, etc.) im Vordergrund stehen, sondern ausschließlich die typischen Effekte, die in einer realen Schaltung auftreten.
- Jedes reale Gatter besitzt verschiedene Verzögerungszeiten
 - technologieabhängig
 - belastungsabhängig (Fan-In, Fan-Out)
 - platinenabhängig
 - leitungsabhängig, etc.

5.4 Dynamisches Verhalten kombinatorischer Schaltungen

- Aus diesem Grund legen wir für die folgenden Betrachtungen eine virtuelle Verzögerungszeit

$$\Delta T_i$$

für jeden Gattertyp i fest.

- Dieses Modell ist zwar sehr grob und wird in heutigen Simulationseinrichtungen nicht mehr verwendet, jedoch lassen sich grundsätzlich dynamische Verhaltensweisen recht anschaulich damit erklären.

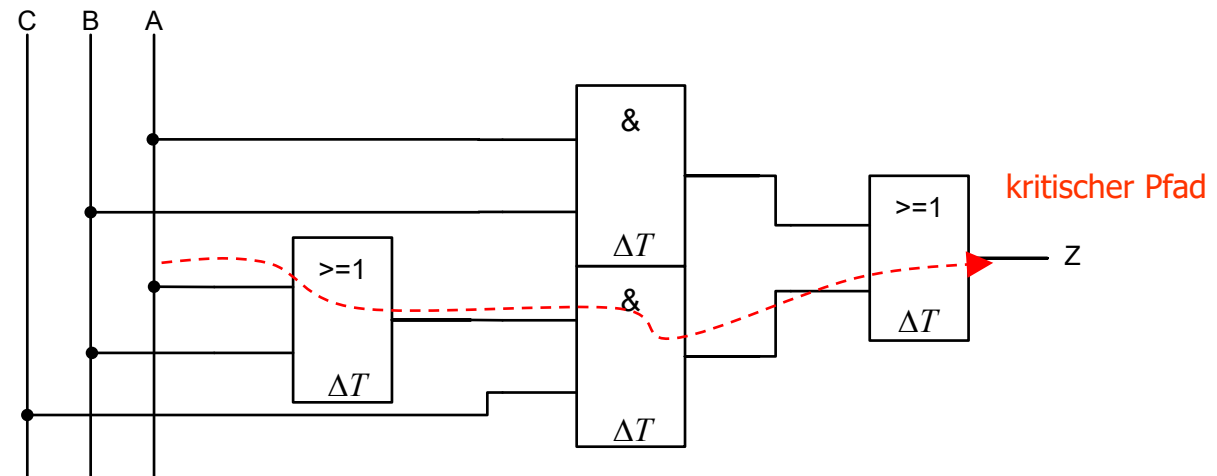
5.4 Dynamisches Verhalten kombinatorischer Schaltungen

- **Aussagen zu realen kombinatorischen Schaltungen**
 - Bei idealen kombinatorischen Schaltungen wurde bis jetzt jede Änderung der Eingangsvariablen *sofort, verzögerungsfrei* für eine (oder mehrere) Ausgangsvariablen umgesetzt.
 - Das Verhalten realer Schaltungen entspricht *nicht* dem o. g. Verhalten, da diese *nicht verzögerungsfrei* sind.
 - Die Verzögerungszeit einer kombinatorischen Schaltung wird von seiner *Stufentiefe und der Gatterverzögerungszeit* selbst bestimmt.

5.4 Dynamisches Verhalten kombinatorischer Schaltungen

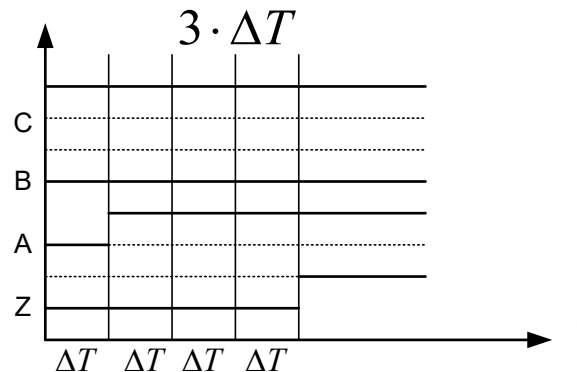
■ Beispiel

- Annahme: $\Delta T_i = \Delta T$



■ Latenzzeit:

$$T_L = \sum_i \Delta T_i$$

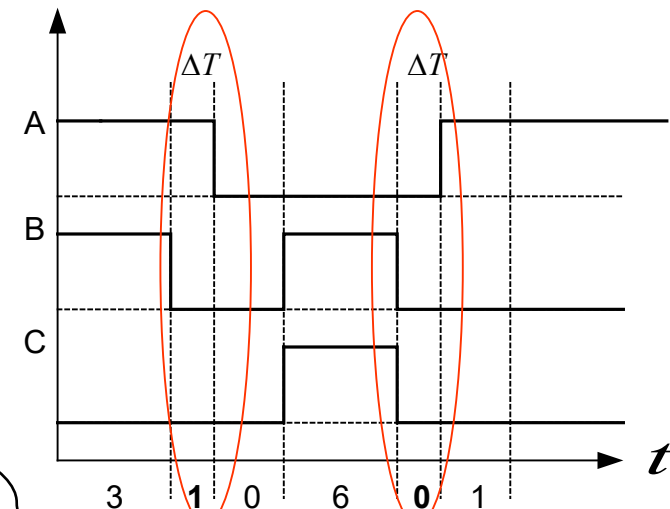
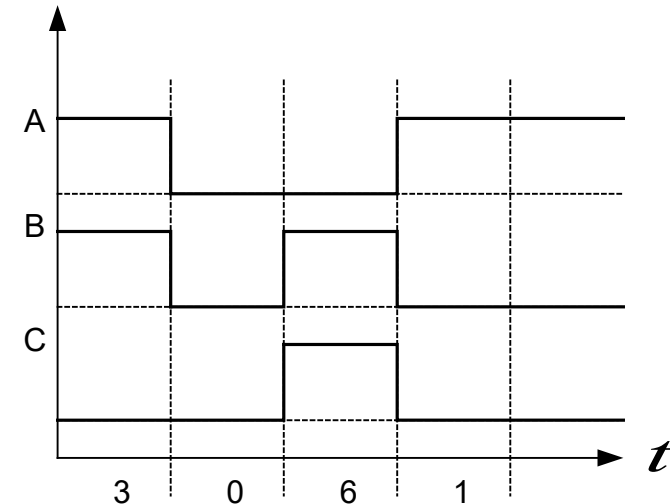


5.4 Dynamisches Verhalten kombinatorischer Schaltungen

- Es ist erkennbar, dass eine gleichzeitige Änderung mehrerer Eingangsvariablen in verschiedenen Zweigen der kombinatorischen Schaltung zu so genannten **Läufen** (*Races*) führt.
- Andererseits ist es denkbar, dass die Eingangsvariablen zu unterschiedlichen Zeitpunkten anliegen. Damit kommt es an den Ausgängen einer Schaltung zu temporären Zustandsänderungen, die durch die Variablenänderungen in den Eingängen hervorgerufen werden.
- Diese unerwünschten Änderungen in den Ausgängen werden als *Glitches* [Glitch (engl.): Störimpuls] bezeichnet. Eine Schaltung die ein derartiges Verhalten aufweist, besitzt einen so genannten *Hazard* [Hazard (engl.): Risiko, Zufall].

5.4.1 Funktionenhazard

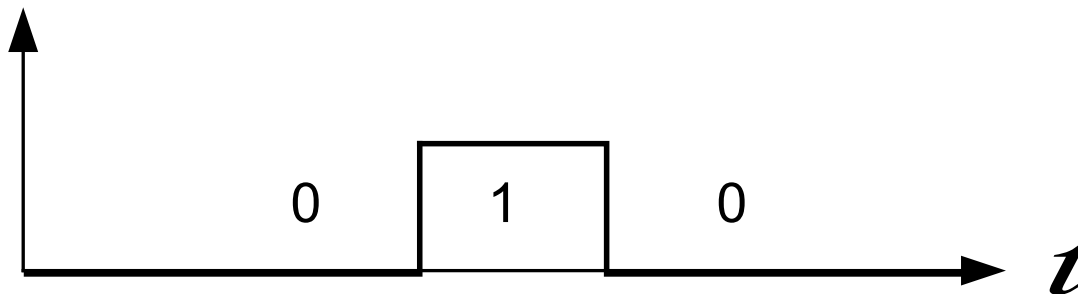
- Wie weiter oben erwähnt, kommt es in der Praxis vor, dass sich Eingangsvariable nicht gleichzeitig ändern. Dieses führt zwangsläufig zu unerwünschten Ausgangssignal-Änderungen, da die logischen Zustände kurzzeitig falsche Werte annehmen.



Hazard

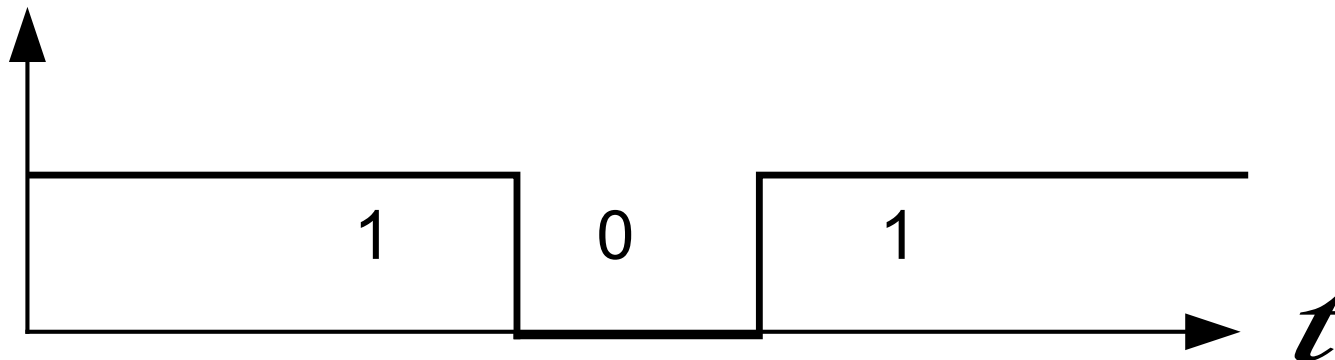
5.4.1 Funktionenhazard

- Unterschiedliche Verzögerungen in den Eingängen führen zu verschiedenen klassifizierbaren Auswirkungsformen von Hazards. Zu nennen sind hier:
- Statischer 0-Hazard
 - Er entsteht, wenn unter idealen Bedingungen eine Ausgangsvariable statisch 0 sein sollte, jedoch zwischenzeitlich ein 1-Glitch auftritt.



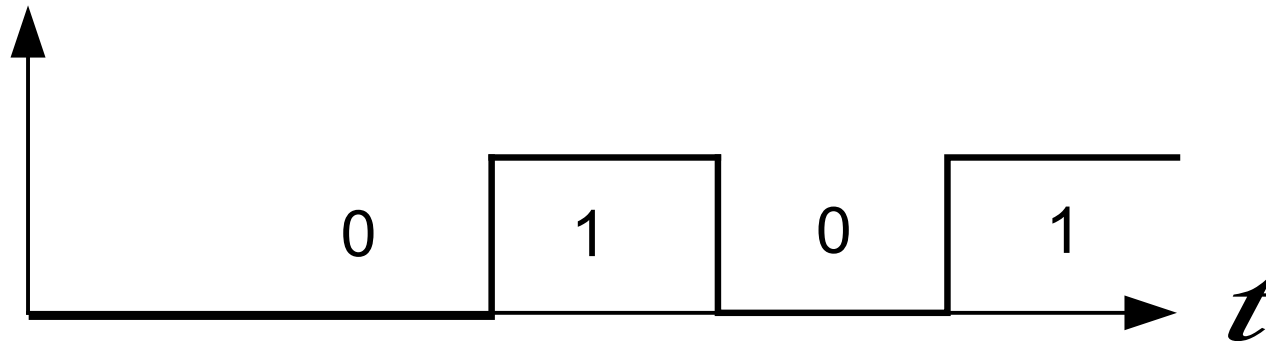
5.4.1 Funktionenhazard

- Statischer 1-Hazard
 - Er entsteht, wenn unter idealen Bedingungen eine Ausgangsvariable statisch 1 sein sollte, jedoch zwischenzeitlich ein 0-Glitch auftritt.



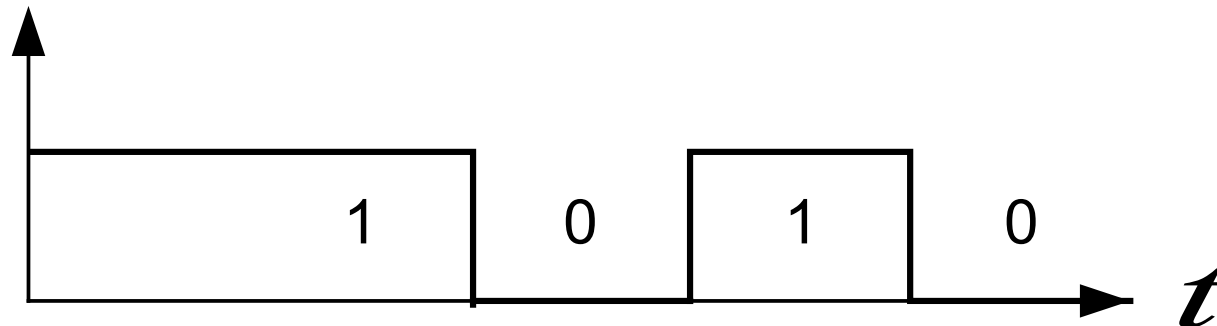
5.4.1 Funktionenhazard

- Dynamischer 0-Hazard
 - Dieser entsteht, wenn bei einem Übergang von 0 nach 1 ein nicht gewollter 0-Glitch auftritt.



5.4.1 Funktionenhazard

- Dynamischer 1-Hazard
 - Dieser entsteht, wenn bei einem Übergang von 1 nach 0 ein nicht gewollter 1-Glitch auftritt.



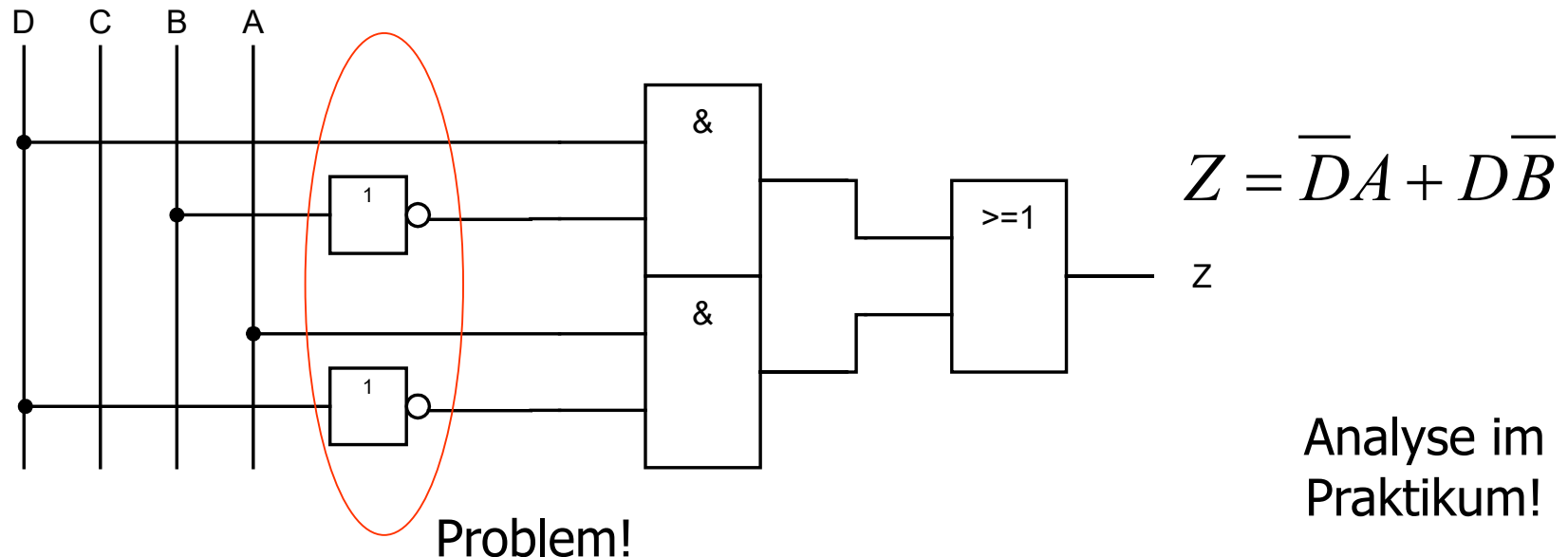
- Mit Hilfe von Speicherschaltungen lassen sich Funktionenhazards weitestgehend vermeiden.

Goldene Regel

Grundsätzlich sollten alle asynchronen Vorgänge in digitalen Schaltungen vermieden werden.

5.4.2 Strukturhazard

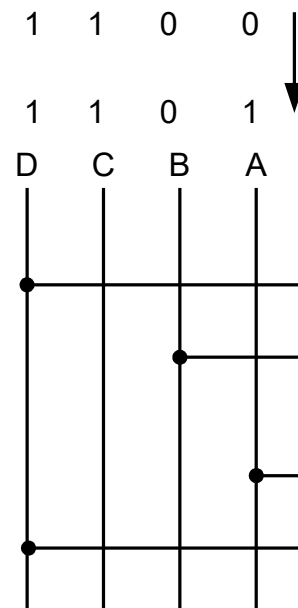
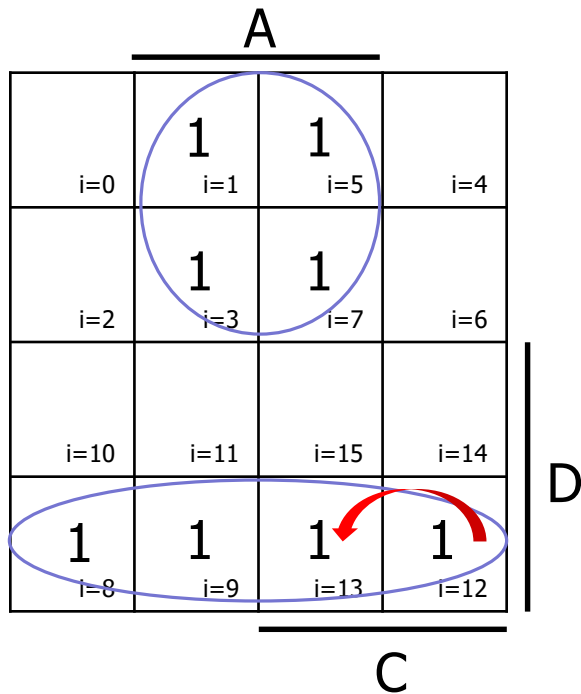
- Aufgrund spezifischer struktureller Eigenschaften kombinatorischer Schaltungen können Signale, die zu einer Ausgangsfunktion verknüpft werden, unterschiedlich verzögert werden. Dieser Sachverhalt führt ebenfalls zu unerwünschten Erscheinungen, die als **Strukturhazards** bezeichnet werden. Wie bei dem Funktionenhazard unterscheidet man auch hier zwischen den bereits angesprochenen vier Typen. Wir betrachten zur Verdeutlichung nur jeweils **eine** Bitstelle.



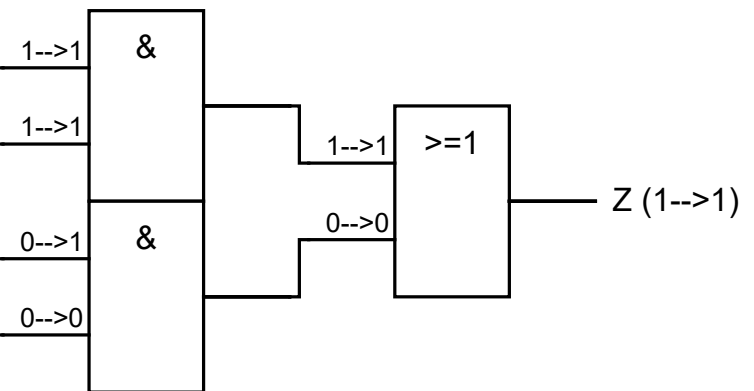
Analyse im
Praktikum!

5.4.2 Strukturhazard

■ K-Plan

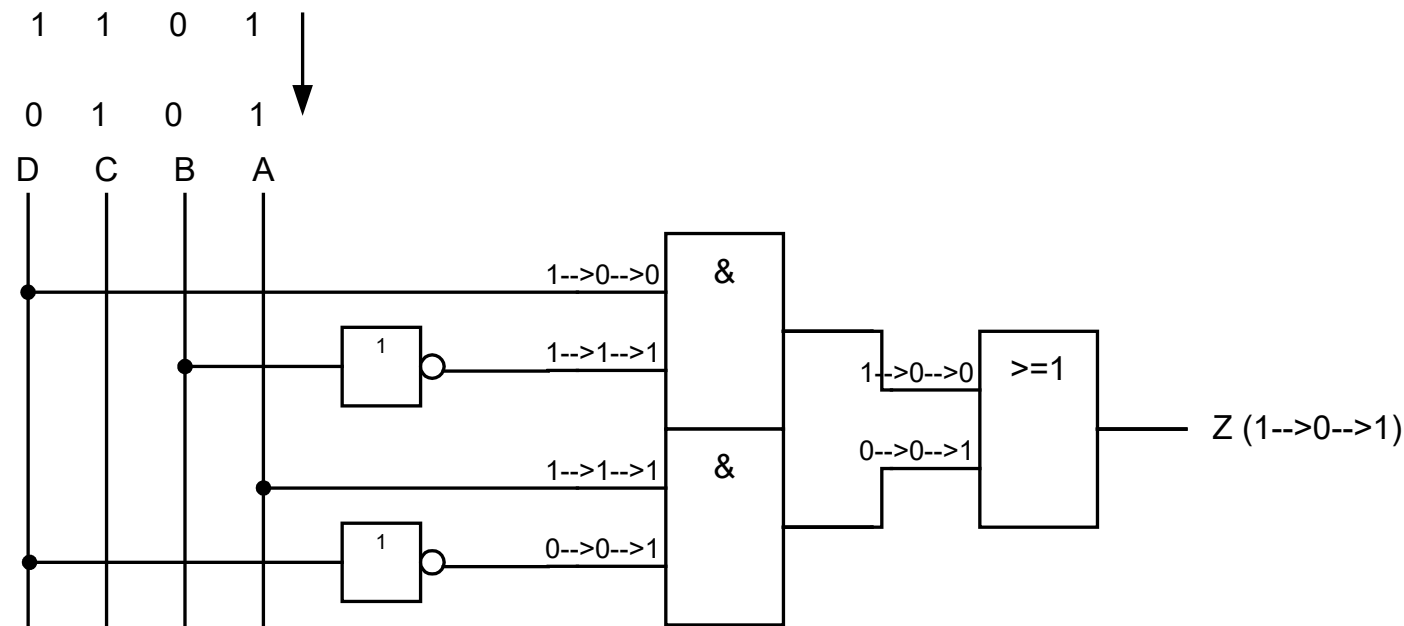
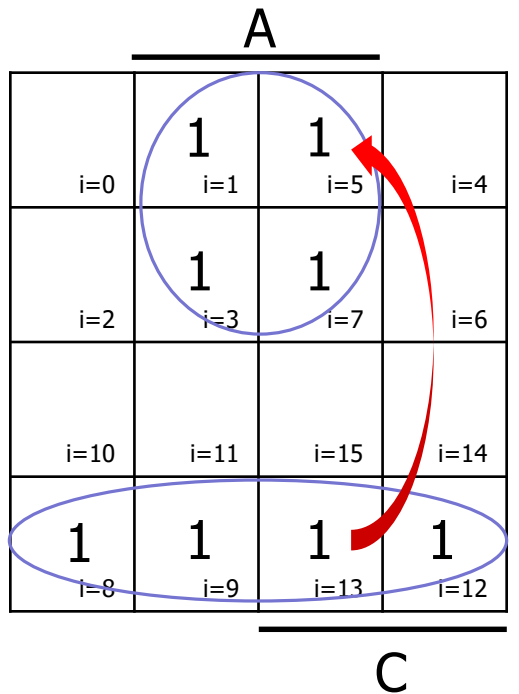


Wir betrachten nun den Übergang von $i = 12$ (1100) nach $i = 13$ (1101). Dazu werden die entsprechenden Werte an die Schaltung angelegt.



5.4.2 Strukturhazard

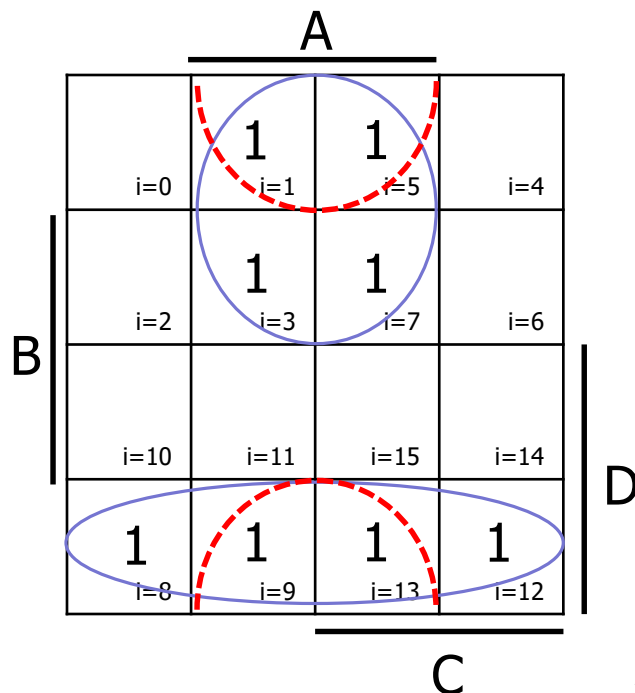
- Anders liegt der Fall, wenn der Übergang von $i = 13$ (1101) nach $i = 5$ (0101) betrachtet wird. Zur Verdeutlichung sei dieses im K-Plan skizziert.



Würde im Übergang das ODER-Gatter nicht schalten, wäre der *1-Hazard* eliminiert.

5.4.2 Strukturhazard

- Betrachtet man den K-Plan ist festzustellen, dass im obigen Fall der Übergang zwischen zwei Primimplikanten der DMF stattgefunden hat. Hieraus folgt sofort, dass mit Hilfe eines **absorbierbaren Primimplikanten (API) der 1-Hazard** eliminiert werden kann. Denn der API sorgt für das nötige statische Verhalten im Übergang.

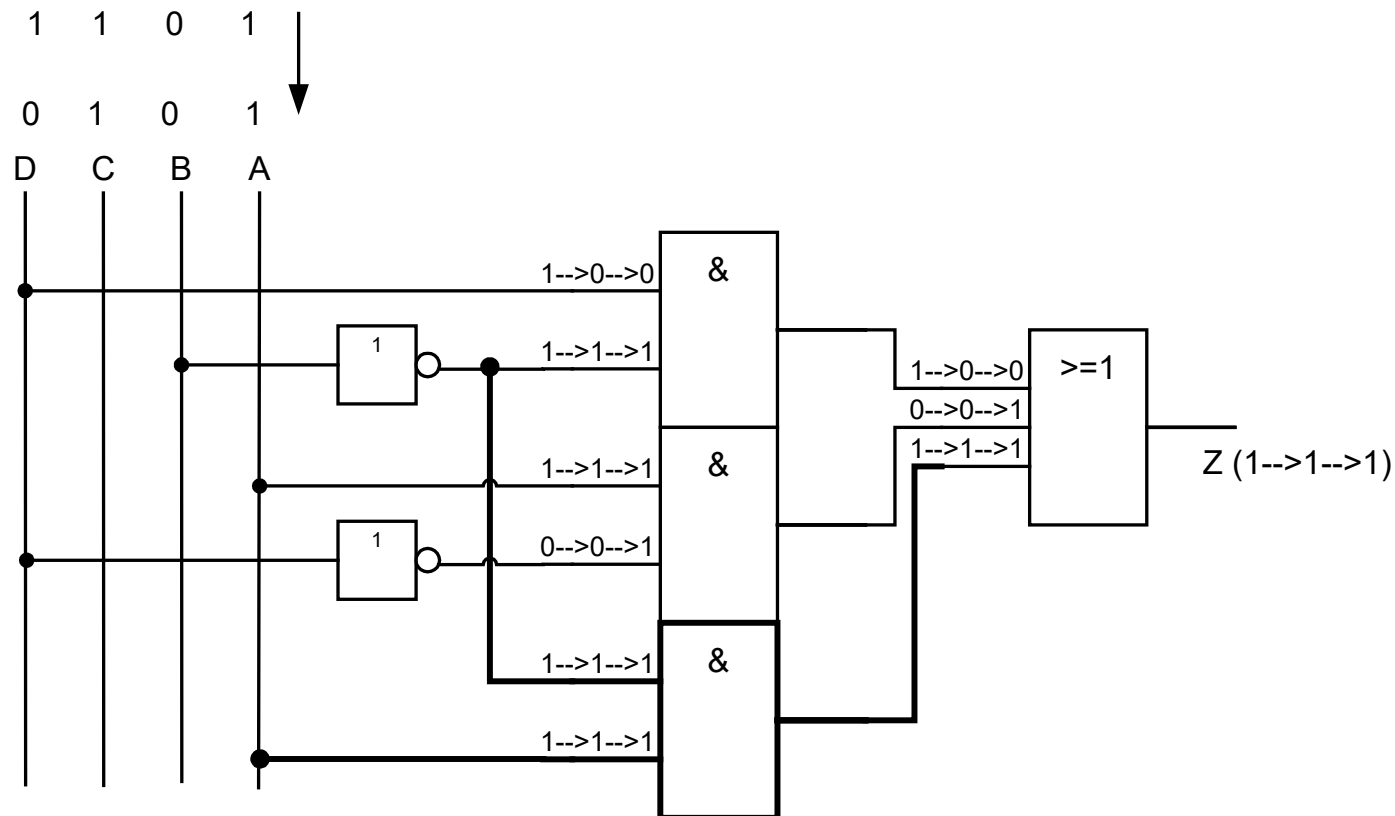


Man spricht in diesem Zusammenhang auch von **Primimplikantenkopplung**. Die zweistufige kombinatorische Schaltung besitzt demnach das folgende Aussehen:

$$Z' = \overline{D}A + D\overline{B} + \overline{B}A$$

5.4.2 Strukturhazard

- Problemlösung
 - Hinzunahme zusätzlicher Gatter



5.4.2 Strukturhazard

■ Regeln zum hazardfreien Entwurf

- Wird eine DMF verwendet, ist sicherzustellen, dass zwei Primimplikanten (PI) durch einen absorbierbaren Primimplikanten (API) teilweise überdeckt werden. Diese Überdeckung sorgt dafür, dass bei einem Übergang der Ausgang einer zweistufigen kombinatorischen Schaltung statisch Einstellenmenge bleibt. Es ist ein UND-Gatter zu verwenden.
- Wird eine KMF verwendet, ist sicherzustellen, dass zwei Primimplikanten (PI) durch einen absorbierbaren Primimplikanten (API) teilweise überdeckt werden. Diese Überdeckung sorgt dafür, dass bei einem Übergang der Ausgang einer zweistufigen kombinatorischen Schaltung statisch bezüglich der Nullstellenmenge bleibt. Es ist ein ODER-Gatter zu verwenden.
- Ein statischer 0-Hazard kann in einer zweistufigen kombinatorischen Schaltung nur auftreten, wenn an einem UND-Gatter eine Variable direkt und negiert anliegt. Hier ist die Verzögerung des Inverters zu berücksichtigen.

5.4.2 Strukturhazard

■ Regeln zum hazardfreien Entwurf

- An dieser Stelle seien einige Anmerkungen gestattet:
 - Für mehrstufige Schaltungen gelten die Aussagen sinngemäß. Es sind verschiedene Methoden in der Literatur bekannt, mit deren Hilfe auch mehrstufige Schaltungen hazardfrei gemacht werden können. Grundsätzlich gilt auch an dieser Stelle unsere *Goldene Regel*.
 - Sie werden sich einerseits gefragt haben, warum wir seinerzeit die Kostenfunktion für zweistufige kombinatorische Schaltungen definiert haben. Andererseits ist Ihnen die eingehende Betrachtung der Implikantenbegriffe zu lang erschienen. Jetzt sehen Sie, dass dieses Vorgehen sinnvoll war, weil wir mittels Implikanten das Verhalten realer Schaltungen besser verstehen. Ebenso war die Einführung des Begriffs **Kosten** sinnvoll, da tatsächlich die Schaltkreiskosten durch hazardfreie Schaltungen steigen.
 - Prinzipiell könnte man eine optimale Schaltung nun nicht mehr nur durch die Anzahl der Gatter und Literale definieren, sondern auch durch die temporäre Stabilität derselben.